```
        {
            it["should earn points for each flight"] = () =>
            {
                member.earnStatusPoints(100);
                member.earnStatusPoints(50);
              member.getStatusPoints().should_be(150);
            };

            it["should upgrade status when enough points are earned"] = () =>
            {
                member.earnStatusPoints(300);
                member.getStatus().should_be(Status.Silver);
            };
        };
    };
};
```

You can use more conventional unit-testing tools like JUnit and NUnit to achieve a similar level of readability by using readable class and method names. The following NUnit test, for example, uses nested classes to organize low-level specifications in a more meaningful way.

```
[TestFixture]
public class WhenUpdatingStatusPoints
{
    public class WhenTheFrequentFlyerAccountIsCreated
    {
        [Test]
        public void it_should_have_zero_points() {...}

        [Test]
        public void it_should_have_Bronze_status() {...}
    }

    public class WhenCumulatingStatusPoints
    {
        [Test]
        public void should_earn_points_for_each_flight() {...}

        [Test]
        public void upgrade_status_level_with_enough_points() {...}
    }
}
```

As illustrated in the Microsoft Visual Studio screenshot in figure 11.12, this makes the test classes read less like a list of test methods and more like a hierarchically organized specification document.

When a code base grows large, it can be hard to find the unit test that illustrates a particular feature. Many teams prefer to organize their unit tests in a package structure that mirrors the application structure. This approach has the advantage of making it easier to find the test class for a particular test, although this advantage is much less compelling with today's development environments, where it's very easy to find all the places in the code base where a particular class has been used.